Dynamic Prototype Learning via Online Hybrid Kalman-Momentum Updates in Continuous Hopfield Networks

1 Executive Summary

Continuous Hopfield networks depend critically on the stability and geometric structure of their internal memory prototypes for associative pattern retrieval. In dynamic representation learning environments, conventional memory update mechanisms face a fundamental dilemma: static, fixed-rate moving averages treat all training periods equally regardless of representation quality, while unconstrained additive updates are highly vulnerable to noisy or deteriorating gradients.

This proposal introduces an online **Hybrid Kalman-Momentum Update Rule** for continuous Hopfield prototype memories. Rather than updating prototypes via a static exponential moving average (EMA), each memory prototype maintains an independent state comprising:

1. **Geometric Momentum Vector (V_j):** An exponential moving average of displacement vectors that smooths spatial trajectories and prevents erratic oscillations in embedding space.
2. **Uncertainty Estimate (P_j):** A dynamic uncertainty value propagated through time with process noise injection.
3. **Accuracy-Gated Kalman Trust (K_j):** A scalar gain derived from running model classification accuracy and prototype uncertainty, throttling memory mutation during periods of high classifier error and accelerating adaptation when representations are reliable.

The entire memory framework is integrated into an online minibatch training workflow with accuracy-guided admission, centered-cosine similarity scoring, and raw prototype readout, ensuring complete gradient isolation from the direct deep classifier.

2 Problem Statement and Theoretical Motivation

Let

$$f_\theta : \mathcal{X} \to \mathbb{R}^D$$

be a deep convolutional feature extractor that maps an input sample $x \in \mathcal{X}$ to an embedding vector $z \in \mathbb{R}^D$, and let

$$g_\phi : \mathbb{R}^D \to \mathbb{R}^C$$

denote a linear classification head producing class logits over C target categories.

A modern continuous Hopfield memory layer stores a set of prototype vectors

$$\{M_j\}_{j=1}^{N} \subset \mathbb{R}^D,$$

organized across C discrete classes. For an incoming query embedding $q \in \mathbb{R}^D$, the associative retrieval operation computes:

$$h = \sum_{j=1}^{N} w_j M_j, \qquad w_j = \text{softmax}\left(\beta \cdot \text{Sim}(q, M_j)\right).$$

where β is the inverse temperature parameter.

Updating M dynamically as f_θ learns introduces three primary failure modes:

1. **The Trajectory Problem (High-Frequency Stuttering):** Updating prototypes directly toward instantaneous candidate vectors causes spatial oscillation. If an individual minibatch contains difficult or atypical samples, prototypes make sharp, discontinuous turns in $\mathbb{R}^D$.

2. **The Trust Problem (Catastrophic Degradation):** Early in training, or following transient distribution shifts, feature representations are noisy. A fixed update rate forces memory prototypes to absorb low-quality representations with the same magnitude as mature, high-confidence representations.

3. **The Softmax Magnitude Distortion:** In inner-product associative memories, unbounded growth or shrinkage of prototype norms $\|M_j\|_2$ artificially skews the effective retrieval temperature

   $$\beta \cdot \|q\|_2 \|M_j\|_2,$$

   leading to degenerate attention concentration.

3 Network Architecture and Direct Classifier Integration

3.1 Dual-Path Evaluation Topology

The architecture comprises two paths sharing the identical encoder f_θ and linear classifier g_ϕ:

- **Direct Path:**
$$x \xrightarrow{f_\theta} z \xrightarrow{g_\phi} \hat{y}_{\text{direct}} = \arg\max g_\phi(z)$$

- **Associative Hopfield Path:**
$$x \xrightarrow{f_\theta} z \xrightarrow{\text{Hopfield}(z;M)} h \xrightarrow{g_\phi} \hat{y}_{\text{hopfield}} = \arg\max g_\phi(h)$$

3.2 Gradient Isolation

To preserve strict experimental control and eliminate confounding optimization dynamics:

- The classifier parameters (θ, ϕ) are trained exclusively via cross-entropy loss evaluated on the direct path:

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(g_\phi(z)_y)}{\sum_{c=1}^{C} \exp(g_\phi(z)_c)}.$$

Prototypes M are maintained as non-differentiable model buffers (`register_buffer`). No gradients flow from the Hopfield retrieval operation back into the feature encoder f_θ.

4 Memory Architecture and State Representation

The memory bank allocates a fixed capacity of K prototype slots per class, yielding a total capacity of

$$N = C \cdot K$$

memory slots.

4.1 Prototype State Variables

Each individual prototype slot $j \in \{1, \ldots, N\}$ maintains:

- **Memory Vector:** $M_j \in \mathbb{R}^D$, storing the raw unnormalized prototype in the encoder's embedding space.
- **Class Label:** $c_j \in \{0, \ldots, C-1\}$, constraining the prototype to represent a single category.
- **Velocity Vector:** $V_j \in \mathbb{R}^D$, tracking historical displacement direction.
- **Uncertainty Scalar:** $P_j \in \mathbb{R}^+$, representing the ongoing variance of the prototype location.
- **Allocation State:** A binary indicator $a_j \in \{0, 1\}$ denoting whether slot j is currently active.

4.2 Global Coordinate Centering

To maximize angular separation and remove global embedding bias, the memory maintains a frozen global center vector $\mu \in \mathbb{R}^D$:

$$\mu = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{x_i \in \mathcal{D}_{\text{train}}} f_\theta(x_i).$$

This is extracted at the conclusion of a designated warm-up training period (E_{warmup} epochs) and held fixed thereafter.

5 Online Minibatch Lifecycle and Admission Protocol

During training, memory updates occur online on a minibatch-by-minibatch basis rather than through batched epoch-end replacement.

5.1 Online Memory Update Workflow

The online memory lifecycle follows the following sequence:

1. Receive minibatch sample (x_i, y_i).
2. Compute direct embedding $z_i = f_\theta(x_i)$.
3. Check whether memory is ready and post-warm-up.
4. If not ready, use direct fallback or bootstrap seeding.
5. Otherwise, retrieve h_i from the memory bank M.
6. Compare direct and Hopfield classification correctness.
7. If the sample is categorized as Harmed, admit it for memory writing.
8. Find the nearest same-class prototype.
9. Merge, add, or skip according to the distance threshold and available class capacity.

5.2 Warm-Up and Bootstrap Seeding

1. **Warm-up Phase ($e \leq E_{\text{warmup}}$):**

   The memory bank remains empty:

   $$a_j = 0 \qquad \forall j.$$

   The direct network trains unimpeded. At the warm-up boundary, center μ is extracted and frozen.

2. **Bootstrap Seeding ($e > E_{\text{warmup}}$ until all classes are represented):**

   Each category c seeks its first directly correct, non-degenerate embedding $f_\theta(x_i)$ with $g_\phi(z_i) = y_i$.

   The embedding is committed to an empty slot as an initial prototype:

   $$M_j \leftarrow z_i, \qquad P_j \leftarrow P_0, \qquad V_j \leftarrow 0, \qquad a_j \leftarrow 1.$$

   While any class remains unseeded, retrieval marks direct fallback.

5.3 Accuracy-Guided Admission Gating

Once all classes possess at least one active prototype, each incoming sample (x_i, y_i) within a minibatch of size B_{train} is categorized using a pre-write snapshot:

$$\text{Category}(i) = \begin{cases} \text{Both Correct}, & \text{if } \hat{y}_i^{\text{direct}} = y_i \text{ and } \hat{y}_i^{\text{hopfield}} = y_i \\ \text{Harmed}, & \text{if } \hat{y}_i^{\text{direct}} = y_i \text{ and } \hat{y}_i^{\text{hopfield}} \neq y_i \\ \text{Helped}, & \text{if } \hat{y}_i^{\text{direct}} \neq y_i \text{ and } \hat{y}_i^{\text{hopfield}} = y_i \\ \text{Both Wrong}, & \text{if } \hat{y}_i^{\text{direct}} \neq y_i \text{ and } \hat{y}_i^{\text{hopfield}} \neq y_i \end{cases}$$

Under the **accuracy-guided admission policy**, an example is eligible to alter memory if and only if:

$$\text{Eligible}(i) \iff \text{Category}(i) = \text{Harmed}.$$

- **Directly incorrect examples are strictly excluded** from memory writes to prevent mislabeled or noisy representations from polluting prototypes.
- **Both-correct examples are skipped** to avoid over-concentrating memory slots on already well-represented patterns.
- Only patterns that the direct CNN classifies correctly, but the associative memory misretrieves, are admitted.

5.4 Nearest Same-Class Distance and Mutation Decision

For an eligible sample with embedding z_i and true class y_i, candidate evaluation is restricted to active prototypes of class y_i:

$$\mathcal{J}(y_i) = \{j \in \{1, \ldots, N\} \mid a_j = 1,\ c_j = y_i\}\,.$$

Centered cosine distance between candidate z_i and active prototype M_j is given by:

$$d(z_i, M_j) = 1 - \left(\frac{z_i - \mu}{\|z_i - \mu\|_2 + \epsilon_{\text{norm}}}\right)^{\top} \left(\frac{M_j - \mu}{\|M_j - \mu\|_2 + \epsilon_{\text{norm}}}\right).$$

The nearest prototype index j^* and minimal distance d^* are determined as:

$$j^* = \arg\min_{j \in \mathcal{J}(y_i)} d(z_i, M_j),$$

$$d^* = d(z_i, M_{j^*}).$$

The action is decided according to merge threshold τ_{merge}:

1. **Merge ($d^* \leq \tau_{\text{merge}}$):**

   The candidate lies within the spatial neighborhood of prototype M_{j^*}. Prototype M_{j^*} absorbs the new evidence via the update rule.

2. **Add ($d^* > \tau_{\textbf{merge}}$ and $|\mathcal{J}(y_i)| < K$):**

   The candidate represents an uncovered sub-mode of class y_i. A free slot is allocated:

$$M_{\text{new}} \leftarrow z_i,$$

$$V_{\text{new}} \leftarrow 0, \qquad P_{\text{new}} \leftarrow P_0, \qquad a_{\text{new}} \leftarrow 1.$$

3. **Skip ($d^* > \tau_{\textbf{merge}}$ and $|\mathcal{J}(y_i)| = K$):**

   The class capacity is fully saturated; the candidate is discarded without memory eviction.

6 Mathematical Formulation of the Update Mechanism

When a candidate embedding z_i triggers a **Merge** on prototype M_j, its state variables (M_j, V_j, P_j) are updated sequentially.

Step 1: Measurement Noise Derivation

Let $A_n \in [0, 1]$ be the current baseline classification accuracy evaluated across clean training data. The observation noise R_n is defined as:

$$R_n = (1 - A_n) + \epsilon_{\text{stab}}.$$

where $\epsilon_{\text{stab}} > 0$ guarantees numerical stability and prevents division by zero. If classifier accuracy is low, observation noise is large ($R_n \to 1$).

Step 2: Kalman Gain Computation

The scalar Kalman gain K_j balances the prototype's internal uncertainty against the current observation noise:

$$K_j = \frac{P_j}{P_j + R_n}.$$

- When $P_j \gg R_n$ (high prototype uncertainty and high model accuracy), $K_j \to 1$.
- When $R_n \gg P_j$ (model accuracy is degraded), $K_j \to 0$, effectively freezing the prototype.

Step 3: Velocity and Momentum Smoothing

The historical velocity V_j accumulates the displacement vector $(z_i - M_j)$ via an exponential moving average with momentum discount factor $\gamma \in [0, 1)$:

$$V_j \leftarrow \gamma V_j + (1 - \gamma)(z_i - M_j).$$

Step 4: Gated Prototype Position Update

The prototype steps along the smoothed velocity direction, scaled by the Kalman gain:

$$M_j \leftarrow M_j + K_j V_j.$$

Step 5: Uncertainty Propagation

The uncertainty is contracted by the incorporated evidence and refreshed with process noise $Q > 0$:

$$P_j \leftarrow (1 - K_j)P_j + Q.$$

Process noise Q ensures that P_j never collapses to zero, enabling the prototype to adapt if representation drift occurs in later training epochs.

7 Comparative Method Ablations

To isolate the individual contributions of momentum trajectory smoothing and Kalman accuracy gating, the experiment formulates four controlled variants sharing the identical training trajectory, data splits, and admission protocol:

1. **Exponential Moving Average (EMA Baseline):**

   Disables both momentum and Kalman gating. Uses fixed scalar learning rate $\alpha_{\text{ema}} \in (0, 1]$:

   $$M_j \leftarrow (1 - \alpha_{\text{ema}})M_j + \alpha_{\text{ema}} z_i.$$

2. **Hybrid Kalman-Momentum (Full Proposed Method):**

   Applies velocity smoothing (V_j) and Kalman gain gating (K_j):

   $$M_j \leftarrow M_j + K_j V_j.$$

3. **Momentum-Only Ablation:**

   Maintains momentum velocity smoothing but fixes the update gate to unity ($K_j \equiv 1$):

   $$M_j \leftarrow M_j + 1.0 \cdot V_j,$$

   $$V_j \leftarrow \gamma V_j + (1 - \gamma)(z_i - M_j).$$

4. **Kalman-Only Ablation:**

   Eliminates momentum history ($V_j \equiv z_i - M_j$) and relies solely on the accuracy-gated Kalman step:

   $$M_j \leftarrow M_j + K_j(z_i - M_j).$$

8 Centered Directional Hopfield Retrieval

To prevent vector norm inflation from altering the softmax attention landscape, similarity calculation is strictly decoupled from value readout.

8.1 Directional Key Normalization

For any query $q \in \mathbb{R}^D$ and active prototype $M_j \in \mathbb{R}^D$:

$$\tilde{q} = \frac{q - \mu}{\|q - \mu\|_2 + \epsilon_{\text{norm}}},$$

$$\tilde{M}_j = \frac{M_j - \mu}{\|M_j - \mu\|_2 + \epsilon_{\text{norm}}}.$$

8.2 Attention Weight Computation

Attention weights $w \in \Delta^{|\text{active}|-1}$ are computed across all currently active prototype slots:

$$w_j = \frac{\exp\left(\beta \cdot \tilde{q}^\top \tilde{M}_j\right)}{\sum_{k \in \text{active}} \exp\left(\beta \cdot \tilde{q}^\top \tilde{M}_k\right)}.$$

Stored class labels c_j do **not** participate in retrieval weighting; attention is entirely label-agnostic.

8.3 Raw Prototype Linear Readout

The retrieved associative representation $h \in \mathbb{R}^D$ is the convex combination of the **raw, unnormalized** prototype vectors:

$$h = \sum_{j \in \text{active}} w_j M_j.$$

Because downstream classifier weights g_ϕ are trained on raw embeddings, feeding raw prototype combinations h preserves the natural activation scale.

9 Evaluation Protocol and Metrics

Evaluation is performed in read-only mode across independent random seeds S.

9.1 Clean Accuracy Metrics

Evaluated on disjoint clean validation and test splits:

- Direct classification accuracy:

$$\text{Acc}_{\text{direct}} = \frac{1}{|\mathcal{D}|} \sum_i \mathbb{I}\left(\hat{y}_i^{\text{direct}} = y_i\right).$$

- Memory retrieval accuracy:

$$\mathrm{Acc}_{\mathrm{hopfield}} = \frac{1}{|\mathcal{D}|} \sum_i \mathbb{I}\left(\hat{y}_i^{\mathrm{hopfield}} = y_i\right).$$

9.2 Corruption Robustness Evaluation

The clean test split is subjected to parameterized corruption transformations:

- **Additive Gaussian Noise:**

$$x' = \mathrm{clip}\left(x + \mathcal{N}(0, \sigma^2), 0, 1\right)$$

  across noise grid Σ.

- **Impulsive Salt-and-Pepper Noise:**

  Pixel replacement with probability $p \in \mathcal{P}$.

- **Spatial Occlusion:**

  Black square cutout of size $s \times s$ pixels across size grid $\mathcal{S}$.

For each corruption family and level, test accuracy is recorded for both the Direct CNN baseline and each memory method.

9.3 Paired Sample Statistics

For each sample, pairwise performance is tabulated:

- Helped:

$$\mathrm{Helped} = \sum_i \mathbb{I}\left(\hat{y}_i^{\mathrm{direct}} \neq y_i \wedge \hat{y}_i^{\mathrm{hopfield}} = y_i\right).$$

- Harmed:

$$\mathrm{Harmed} = \sum_i \mathbb{I}\left(\hat{y}_i^{\mathrm{direct}} = y_i \wedge \hat{y}_i^{\mathrm{hopfield}} \neq y_i\right).$$

- Accuracy Difference:

$$\Delta\mathrm{Acc} = \frac{\mathrm{Helped} - \mathrm{Harmed}}{|\mathcal{D}|}.$$

10 Summary of Hyperparameters

Symbol	Description	Mathematical Role
C	Number of target classes	Categorical partitioning
D	Embedding representation dimension	Feature vector length
K	Capacity limit of prototypes per class	Maximum allowable memory slots per class
E_{total}	Total training epochs	Training duration
E_{warmup}	Initial warm-up epochs	Direct CNN feature stabilization before memory activation
B_{train}	Training minibatch size	Optimization and online stream chunk size
B_{eval}	Evaluation minibatch size	Inference throughput control
η	Optimizer learning rate	Step size for classifier parameters (θ, ϕ)
α_{ema}	Static moving average rate	Baseline EMA merge weighting
γ	Momentum discount coefficient	Velocity exponential decay rate in $[0, 1)$
Q	Base process noise	Uncertainty injection rate per update step
P_0	Initial prototype uncertainty	Prior variance assigned to new prototype slots
ϵ_{stab}	Numerical stability epsilon	Divisor guard for measurement noise R_n
ϵ_{norm}	Vector normalization epsilon	Divisor guard for unit vector normalization
τ_{merge}	Centered cosine merge threshold	Distance boundary separating merge versus new slot allocation
β	Hopfield inverse temperature	Softmax scaling factor controlling retrieval sharpness

Table 1: Summary of model and training hyperparameters.